# LAPORAN PRAKTIKUM

# ALGORITMA DAN PEMOGRAMAN

# ABSTRACT DATA TYPE (ADT)


**disusun Oleh:**

**FILZI JELILA INDA ROBBANI**

**(2511533019)**

**Dosen Pengampu:**

**Dr. WAHYUDI, S.T, M.T**

**Asisten Praktikum:**

**ZAHRAN AZARIA ANVAYA**

**FAKULTAS TEKNOLOGI INFORMASI**

**DEPARTEMEN INFORMATIKA**

**UNIVERSITAS ANDALAS**


**2026**

# KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Algoritma dan Pemrograman kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan mengenai Abstract Data Type (ADT) dalam bahasa pemrograman Java.

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

# DAFTAR ISI

**KATA PENGANTAR** ..... **ii**
**DAFTAR ISI** ..... **iii**
**BAB I PENDAHULUAN** ..... **4**
1.1 Latar Belakang ..... 4
1.2 Tujuan ..... 4
1.3 Manfaat ..... 5
**BAB II PEMBAHASAN** ..... **12**
2.1 Langkah Kerja Praktikum ..... 12
**2.1.1 Membuat kelas pertama (Jam)** ..... 12
**2.1.2 Membuat program kedua (Driver/Pemanggil kelas Jam)** ..... 14
**2.1.3 Membuat program ketiga (Driver2)** ..... 15
2.2 Tugas Praktikum ..... 18
**2.1.1 Membuat kelas ADT Mobil** ..... 18
**2.2.2 Membuat class Driver:** ..... 21
**BAB III PENUTUP** ..... **26**
**3.1 Kesimpulan** ..... **26**
**DAFTAR PUSTAKA** ..... **27**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam ilmu komputer yang berperan penting dalam pengelolaan dan penyimpanan data secara efisien. Dalam implementasinya, struktur data tidak dapat dipisahkan dari konsep Abstract Data Type (ADT), yaitu suatu model yang mendefinisikan data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan bagaimana implementasinya secara detail.

Pada tahap awal praktikum ini, mahasiswa diperkenalkan dengan konsep ADT sebagai dasar dalam memahami berbagai jenis struktur data. ADT membantu dalam merancang suatu sistem secara terstruktur dengan memisahkan antara definisi data dan implementasinya.

Dengan memahami konsep ADT, mahasiswa diharapkan mampu mendefinisikan tipe data secara abstrak melalui pembuatan konstruktor, method setter, getter, serta fungsi atau prosedur lain yang mendukung. Hal ini menjadi dasar penting dalam pemrograman berorientasi objek, khususnya dalam bahasa Java.

Oleh karena itu, pada praktikum pekan pertama ini difokuskan pada pemahaman dan implementasi Abstract Data Type (ADT) agar mahasiswa memiliki dasar yang kuat dalam merancang dan mengembangkan struktur data secara sistematis dan efisien.

## 1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

a. Memahami konsep dasar Abstract Data Type (ADT).
b. Mengetahui cara mendefinisikan ADT dalam bahasa pemrograman Java.

c. Mampu memahami pengimplementasian konstruktor, setter, dan getter dalam suatu kelas.
d. Mampu membuat fungsi atau prosedur yang sesuai dalam ADT.
e. Melatih kemampuan dalam merancang struktur program yang sistematis dan terstruktur.

## 1.3 Manfaat

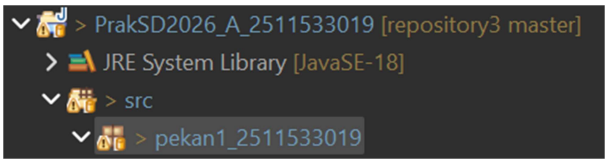
a. Memberikan pemahaman dasar mengenai konsep Abstract Data Type (ADT) dalam struktur data.
b. Membantu mahasiswa dalam memahami cara merancang tipe data secara abstrak sebelum diimplementasikan ke dalam program.
c. Melatih kemampuan dalam membuat program menggunakan konsep pemrograman berorientasi objek, seperti penggunaan kelas, konstruktor, setter, dan getter.
d. Meningkatkan kemampuan dalam menyusun logika program yang terstruktur dan sistematis.
e. Menjadi dasar pengetahuan untuk memahami materi struktur data selanjutnya.

# BAB II
# PEMBAHASAN

## 2.1 Langkah Kerja Praktikum

### 2.1.1 Membuat kelas pertama (Jam)

1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah folder baru dengan nama PrakSD2026_A_2511533019 dan buat package baru dengan mengklik kanan di folder src. Setelah itu beri nama “pekan1_2511533019”.




2) Setelah itu klik kanan package pekan4 tsb dan pilih “New”, pilih class. Buat nama “Jam_2511533019”.
3) Lalu, masukkan syntax seperti berikut.

Jam_2511533019.java

```java
package pekan1_2511533019;

public class Jam_2511533019 {
    private int hh;
    private int mm;
    private int ss;

    //validator
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h<= 23) && (0<= m && m <= 59) && (0 <= s && s <= 59);
    }
    //konstruktor
    public Jam_2511533019(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }
    //selektor
    public int getHour() {return hh;}
    public int getMinute() {return mm;}
    public int getSecond() {return ss;}
    //mutator
    public void setHour (int h) {this.hh = h;}
    public void setMinute (int m) {this.mm = m;}
    public void setSecond (int s) {this.ss = s;}
    //Konversi
    public int toSeconds() {return hh *3600 +mm * 60 + ss;}
    public static Jam_2511533019 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %=24 * 3600;
        int h =total / 3600; total %= 3600;
        int m =total / 60; int s = total % 60;
        return new Jam_2511533019(h, m, s);
    }
```

```
33      //relasional
34      public int compareTo(Jam_2511533019 other) { return Integer.compare(this.toSeconds(), other.toSeconds());}
35      public boolean equals(Object o) {
36          if(!(o instanceof Jam_2511533019 j))return false;
37          return hh == j.hh & mm == j.mm && ss == j.ss;
38      }
39      public int hashCode() {return java.util.Objects.hash(hh, mm, ss);
40      }
41
42      //aritmatika
43      public Jam_2511533019 plus (Jam_2511533019 other) {return fromSeconds(this.toSeconds()+ other.toSeconds());}
44      public Jam_2511533019 minus (Jam_2511533019 other) {return fromSeconds(Math.floorMod(this.toSeconds()-other.toSeconds(), 24*3600));}
45      public Jam_2511533019 nextSeconds () {return  fromSeconds(this.toSeconds()+1); }
46      public Jam_2511533019 nextNSeconds (int n) {return fromSeconds(this.toSeconds() + Math.max(0, n));}
47      public Jam_2511533019 prevSecond() { return fromSeconds (Math.floorMod(this.toSeconds()- 1, 24*3600));}
48      public Jam_2511533019 prevNSecond(int n){ return fromSeconds (Math.floorMod(this.toSeconds()- Math.max(0, n), 24*3600));}
49
50      //durasi (detik)--bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
51      public static int durasiDetik(Jam_2511533019 jaw, Jam_2511533019 jakh) {return jakh.toSeconds() - jaw.toSeconds();}
52      //overriding ke string
53      public String toString() {return String.format("%02d:%02d:%02d", hh, mm, ss);}
54  }
55
```

Class Jam tersebut digunakan untuk mempresentasikan waktu dalam format jam, menit dan detik. Di dalam class terlihat mendeklarasikan 3 variabel yaitu hh, mm, dan ss dengan tipe data int, yang mana var tsb sebagai tempat penyimpanan nilai jam, menit dan detik.

Selanjutnya, menggunakan method isValid() terlihat pada baris yang berfungsi sebagai validator untuk mengecek apakah nilai jam, menit, dan detik yang dimasukkan sudah sesuai dengan aturan waktu. Kondisi yang digunakan adalah jam harus berada di antara 0–23, sedangkan menit dan detik berada di antara 0–59.

Kemudian, membuat konstruktor yang digunakan untuk menginisialisasi nilai jam, menit, dan detik saat objek dibuat.

Setelah itu, juga membuat method selektor atau getter yaitu getHour(), getMinute(), dan getSecond() yang berfungsi untuk mengambil nilai jam, menit, dan detik dari objek. Selain itu ada juga membuat method mutator atau setter yaitu setHour(), setMinute(), dan setSecond() yang digunakan untuk mengubah nilai jam, menit, dan detik.

Pada bagian konversi, dibuat method toSeconds() yang berfungsi untuk mengubah waktu menjadi total detik. Selain itu, terdapat method fromSeconds() yang digunakan untuk mengubah total detik kembali menjadi objek waktu (jam, menit, detik).

Kemudian, pada bagian relasional, menggunakan method compareTo() untuk membandingkan dua objek waktu berdasarkan jumlah detiknya. Terus ada method equals() untuk mengecek

apakah dua objek waktu memiliki nilai yang sama, serta hashCode() untuk menghasilkan nilai hash dari objek.

Selanjutnya, pada bagian aritmatika, dibuat beberapa method operasi seperti plus() untuk menjumlahkan dua waktu, minus() untuk mengurangkan dua waktu, nextSecond() untuk menambah satu detik, serta nextNSeconds() untuk menambah beberapa detik. Selain itu, terdapat juga method prevSecond() dan prevNSecond() yang digunakan untuk mengurangi waktu.

Pada bagian durasi, menggunakan method durasiDetik() yang berfungsi untuk menghitung selisih waktu dalam satuan detik antara dua objek waktu.

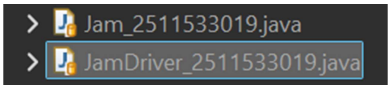
Terakhir, terlihat ada override pada method toString() yang digunakan untuk menampilkan waktu dalam format HH:MM:SS agar lebih mudah dibaca.

4) Class Jam ini tidak dapat di run secara langsung karena kelas ini hanya berfungsi untuk merepresentasikan data waktu beserta operasi-operasi yang dapat dilakukan terhadapnya, seperti konversi, perbandingan, dan operasi aritmatika. Oleh karena itu, class ini lebih berperan sebagai *blueprint* atau template, bukan sebagai program utama yang berdiri sendiri.

   Untuk menjalankan dan menguji class ini, diperlukan kelas lain yang memiliki method main(). Yang memanggil kelas Jam agar bisa untuk melihat hasil programnya.

### 2.1.2 Membuat program kedua (Driver/Pemanggil kelas Jam)

1) Klik kanan package pekan1 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “JamDriver_2511533019.”




2) Masukkan syntax seperti berikut.

```java
Jam_2511533019.java   JamDriver_2511533019.java
1  package pekan1_2511533019;
2
3  public class JamDriver_2511533019 {
4
5      public static void main(String[] args) {
6          // TODO Auto-generated method stub
7          Jam_2511533019 a = new Jam_2511533019(23,59,50);
8          Jam_2511533019 b = new Jam_2511533019(0, 0, 15);
9          System.out.println("a     = "+ a);
10         System.out.println("b     = "+ b);
11         System.out.println("a+b   = " + a.plus(b));
12         System.out.println("next 20s = " + a.nextNSeconds (20)) ;
13         System.out.println("durasi (a,b) = " + Jam_2511533019.durasiDetik(a, b));
14         System.out.println("a.compareTo(b) = " + a. compareTo (b));
15
16     }
17
18 }
```

Pada kode program di atas, class JamDriver_2511533019 digunakan sebagai program utama untuk menguji class Jam_2511533019.

Di dalam method main(), dibuat dua objek yaitu **a** dengan nilai 23,59,50 dan **b** dengan nilai 00,00,15. Kemudian, ditampilkan nilai kedua objek menggunakan System.out.println(), di mana method toString() akan otomatis digunakan untuk menampilkan format waktu.

Selanjutnya, saya menguji beberapa method seperti plus() untuk menjumlahkan waktu, nextNSeconds() untuk menambah detik, durasiDetik() untuk menghitung selisih waktu, serta compareTo() untuk membandingkan dua waktu.

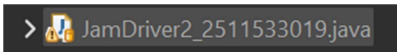
Dengan adanya driver ini, seluruh fungsi pada class Jam_2511533019 dapat di run dan diuji.

3) Berikut tampilan output apabila program di run.

```
Console
<terminated> JamDriver_2511533019
a     = 23:59:50
b     = 00:00:15
a+b   = 00:00:05
next 20s = 00:00:10
durasi (a,b) = -86375
a.compareTo(b) = 1
```

### 2.1.3 Membuat program ketiga (Driver2)

1) Klik kanan package pekan4 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “JamDriver2_2511533019.”



2) Lalu buat/masukkan syntax seperti berikut.

```java
Jam_2511533019.java   JamDriver_2511533019.java   JamDriver2_2511533019.java

package pekan1_2511533019;
import java.util.Scanner;
public class JamDriver2_2511533019 {

    private static Scanner sc;
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver objek Jam ===");
        //1. input jam pertama
        System.out.println("\n--- Input jam 1 ---");
        Jam_2511533019 j1 = buatJamDariInput(input);

        //2.input jam kedua
        System.out.println ("\n--- Input Jam 2 ---");
        Jam_2511533019 j2 = buatJamDariInput(input);

        //3.mnmpilkan data
        System.out.println("\n--- hasil operasi ---");
        System.out.println("Jam 1 (String)     : " +j1.toString());
        System.out.println("Jam 2 (String)     : " +j2.toString());
        System.out.println("Jam 1 dalam detik  : " +j1.toSeconds());
        System.out.println("Jam 2 dalam detik  : " +j2.toSeconds());

        //4. operasi relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0 ) {
            System.out.println("Status        : Jam 1 lebih lambat (setelah) Jam 2");
        }else if (perbandingan <0 ) {
            System.out.println("Status        : Jam 1 lebih awal (sebelum) Jam 2");
        }else {
            System.out.println("Status        : Jam 1 dan Jam 2 sama persis");
        }

```

```java
Jam_2511533019.java   JamDriver_2511533019.java   JamDriver2_2511533019.java

        //5. operasi aritmatika
        System.out.println ("Durasi (J1 ke J2)     : " + Jam_2511533019.durasiDetik(j1, j2)+ " detik");

        Jam_2511533019 jNext = j1.nextSeconds();
        System.out.println("Jam 1 Detik berikutnya : " +jNext);

        Jam_2511533019 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik sebelumnya : " +jPrev);

        //6. operasi penjumlahan jam
        Jam_2511533019 jHasilPlus= j1.plus(j2);
        System.out.println("Hasil J1+J2        : "+ jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */

    private static Jam_2511533019 buatJamDariInput(Scanner sc) {
        // TODO Auto-generated method stub
        int h, m, s;
        while (true) {
        System.out.print ("Masukkan Jam (0-23) : ");
        h = sc.nextInt();
        System.out.print ("Masukkan Menit (0-59) : ");
        m = sc.nextInt ();
        System.out.print ("Masukkan Detik (0-59) : ");
        s = sc.nextInt ();

```

```java
        //memanggil method static isValid dari kelas Jam
        if (Jam_2511533019.isValid(h, m , s)) {
            return new Jam_2511533019 (h, m , s);
        } else {
        System.out.println("[Error] Input tidak valid! Silahkan ulangi. \n");
        }
        }
    }

}
```

Pada kode program di atas, terlihat membuat class JamDriver2_2511533019 sebagai program utama yang digunakan untuk menguji class Jam_2511533019 dengan input dari user.

Di dalam method main(), digunakan Scanner untuk membaca input jam, menit, dan detik dari user. Kemudian, saya membuat dua

objek yaitu jl dan j2 melalui method buatJamDarilnput(, yang berfungsi untuk melakukan input sekaligus validasi agar nilai yang dimasukkan sesuai (jam 0-23, menit dan detik 0-59).

Selanjutnya, program menampilkan data jam dalam bentuk string dan dalam satuan detik menggunakan method toString0 dan toSeconds().

Pada bagian percabangan if-else, dibandingkan kedua waktu menggunakan compareTo() untuk menentukan apakah jam pertama lebih awal, lebih lambat, atau sama dengan jam kedua.

Kemudian, juga menampilkan beberapa operasi seperti durasiDetik( untuk menghitung selisih waktu, nextSeconds( dan prevSecond() untuk melihat detik berikutnya dan sebelumnya, serta plus() untuk menjumlahkan dua waktu.

Dengan adanya driver ini, program dapat menerima input dari user dan menguji seluruh fungsi pada class Jam_2511533019 secara lebih interaktif.

3) Run program hingga keluar output seperti berikut, apabila terjadi error cek kembali syntax dan perbaiki kesalahan.

```
Console
<terminated> JamDriver2_2511533019 [Java Application] C:\Users\Filzi J

--- Input jam 1 ---
Masukkan Jam (0-23) : 2
Masukkan Menit (0-59) : 3
Masukkan Detik (0-59) : 5

--- Input Jam 2 ---
Masukkan Jam (0-23) : 6
Masukkan Menit (0-59) : 2
Masukkan Detik (0-59) : 5

--- hasil operasi ---
Jam 1 (String)      : 02:03:05
Jam 2 (String)      : 06:02:05
Jam 1 dalam detik   : 7385
Jam 2 dalam detik   : 21725
Status         : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)      : 14340 detik
Jam 1 Detik berikutnya : 02:03:06
Jam 1 Detik sebelumnya : 02:03:04
Hasil J1+J2         : 08:05:10

Program Selesai.
```

## 2.2 Tugas Praktikum

### 2.1.1 Membuat kelas ADT Mobil

1) Buka Apl Eclipse IDE, kemudian klik kanan pada package Pekan1
2) Buat class baru pada package Pekan1 dengan nama ADTMobil_2511533019

3) Lalu buat kode java yang berisi atribut dan berbagai method seperti berikut :

```
1) package pekan1_2511533019;
2)
3) public class ADTMobil_2511533019 {
4)     private String nama;
5)     private int tahun;
6)     private int cc;
```

```
7)     private double harga;
8)     private String merk;
9)
10)    //Konstruktor
11)    public ADTMobil_2511533019(String nama, int
   tahun, int cc, double harga, String merk) {
12)        this.nama = nama;
13)        this.tahun = tahun;
14)        this.cc = cc;
15)        this.harga = harga;
16)        this.merk = merk;
17)    }
18)
19)    //Selektor
20)    public String getNama() { return nama; }
21)    public int getTahun() { return tahun; }
22)    public int getCc() { return cc; }
23)    public double getHarga() { return harga; }
24)    public String getMerk() { return merk; }
25)
26)    // Mutator
27)    public void setNama(String nama) { this.nama =
       nama; }
28)    public void setTahun(int tahun) { this.tahun =
       tahun; }
29)    public void setCc(int cc) { this.cc = cc; }
30)    public void setHarga(double harga) { this.harga =
       harga; }
31)    public void setMerk(String merk) { this.merk =
       merk; }
32)
33)    // Hitung pajak
34)    public double hitungPajak() {
35)                return harga * 0.02;
36)        }
37)       // Bandingin harga
38)     public int compareHarga(ADTMobil_2511533019
   other) {
```

```
39)            return Double.compare(this.harga,
   other.harga);
40)     }
41)
42)   // Override ke string
43)     public String toString() {
44)           return "Nama: " + nama +
45)                  ", Tahun: " + tahun
46)                  ", CC: " + cc +
47)                  ", Harga: " + harga +
48)                  ", Merk: " + merk;
49)     }
50) }
```

Pada Pada class ini dibuat beberapa atribut yang digunakan untuk menyimpan data mobil, yaitu nama bertipe String untuk menyimpan nama mobil, tahun bertipe int untuk menyimpan tahun produksi, cc bertipe int untuk menyimpan kapasitas mesin, harga bertipe double untuk menyimpan nilai harga mobil, dan merk bertipe String untuk menyimpan merek mobil. Semua atribut dibuat dengan access modifier private agar tidak dapat diakses langsung dari luar class, sehingga keamanan data lebih terjaga .

**Konstruktor** dibuat untuk menginisialisasi nilai awal dari atribut ketika objek mobil dibuat. Konstruktor menerima parameter sesuai dengan atribut yang dimiliki, kemudian nilai parameter tersebut disimpan ke dalam atribut menggunakan keyword this. Dengan adanya konstruktor ini, setiap objek mobil yang dibuat akan langsung memiliki data sesuai dengan input yang diberikan oleh user tanpa perlu mengisi atribut satu per satu secara manual.

Method **getter** dan **setter** dibuat untuk mengakses dan mengubah nilai atribut secara aman. Getter digunakan untuk mengambil nilai atribut, sedangkan setter digunakan untuk

mengubah nilai atribut. Penggunaan method ini penting karena atribut bersifat private, sehingga tidak dapat diakses secara langsung dari luar class. Dengan demikian, data tetap terlindungi dan hanya dapat dimanipulasi melalui method yang telah disediakan.

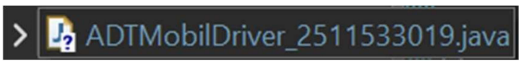
Selain getter dan setter, ditambahkan method lain yaitu method untuk menghitung pajak mobil. Method ini dibuat untuk menghitung dengan cara mengalikan harga mobil dengan persentase pajak sebesar 2%.

Method **toString()** digunakan untuk mengubah data objek mobil menjadi bentuk string agar mudah ditampilkan. Dengan melakukan override pada method ini, seluruh atribut mobil dapat ditampilkan dalam satu baris secara rapi ketika objek dicetak menggunakan System.out.println(). Hal ini mempermudah dalam proses penampilan data kepada user.

Class ini tidak dapat di run karna apabila di run tidak akan menghasilkan ouput apapun. Hal ini dikarenakan class ini hanya berfungsi sebagai blueprint yang berisi atribut dan method untuk mendefinisikan objek mobil, tanpa memiliki method utama sebagai titik awal eksekusi program. Oleh karena itu class ini harus dipanggil melalui class baru (class Driver) agar dapat dirun(menghasilkan suatu output)

### 2.2.2 Membuat class Driver:

1) Klik kanan pada package Pekan1 lalu klik new dan bikin class baru dengan nama ADTMobilDriver_2511533019.




2) Lalu buat/ketik kode program java seperti berikut.

```
1) package pekan1_2511533019;
2) import java.util.Scanner;
3) import java.util.ArrayList;
```

```java

public class ADTMobilDriver_2511533019 {

public static void main(String[] args) {
        // TODO Auto-generated method stub
        Scanner input = new Scanner(System.in);
         ArrayList<ADTMobil_2511533019> dataMobil = new ArrayList<>();

        System.out.println("=== Program ADT Mobil ===");

         // input mobil
        System.out.print("Masukkan jumlah mobil: ");
       int n = input.nextInt();
  input.nextLine();

  for (int i = 0; i < n; i++) {
      System.out.println("\nData Mobil ke-" + (i+1));

      System.out.print("Nama Mobil: ");
      String nama = input.nextLine();

      System.out.print("Tahun: ");
      int tahun = input.nextInt();

      System.out.print("cc: ");
      int cc = input.nextInt();

      System.out.print("Harga: ");
      double harga = input.nextDouble();
      input.nextLine();

      System.out.print("Merk: ");
      String merk = input.nextLine();

      ADTMobil_2511533019 m = new
```

```java
ADTMobil_2511533019(nama, tahun, cc, harga, merk);
            dataMobil.add(m);
        }

        // tampilin data
        System.out.println("\n=== Daftar Mobil ===");
        for (ADTMobil_2511533019 m : dataMobil) {
            System.out.println(m);
            System.out.println("Pajak: " + m.hitungPajak());
            System.out.println("    ");
        }

        // hapus mobil
        System.out.print("\nMasukkan no urutan mobil yang ingin dihapus (mulai dari 0): ");
        int idx = input.nextInt();

        if (idx >= 0 && idx < dataMobil.size()) {
           dataMobil.remove(idx);
           System.out.println("Mobil berhasil dihapus.");
        else {
           System.out.println("Index tidak valid.");
        }

        // tampilin ulang
        System.out.println("\n=== Data Setelah Hapus ===");
        for (ADTMobil_2511533019 m : dataMobil) {
            System.out.println(m);
        }

        input.close();
    }

}
```

Seperti yang telah disebutkan sebelumnya class Driver digunakan untuk menjalankan program serta mengelola data mobil yang telah dibuat pada class ADT yang telah dibuat sebelumnya (ADTMobil_2511533019).

Pada class ini digunakan sebuah ArrayList dengan tipe data objek Mobil yang berfungsi untuk menyimpan banyak data mobil. ArrayList dipilih karena bersifat dinamis sehingga jumlah data dapat bertambah dan berkurang sesuai kebutuhan.

Program meminta user untuk menginputkan data mobil seperti nama, tahun, cc, harga, dan merk-nya melalui input keyboard. Setelah data dimasukkan, program membuat objek mobil menggunakan konstruktor, kemudian menyimpannya ke dalam ArrayList menggunakan method add(). Data mobil yang telah disimpan kemudian ditampilkan menggunakan perulangan . Setiap objek mobil diambil dari ArrayList dan ditampilkan menggunakan method toString() sehingga seluruh informasi mobil dapat dilihat dengan jelas.

Disini saya menggunakan fitur untuk menghapus data mobil berdasarkan index yang dimasukkan oleh user. Index merupakan posisi data dalam ArrayList yang dimulai dari 0. Jika index valid, maka data mobil akan dihapus menggunakan method remove().

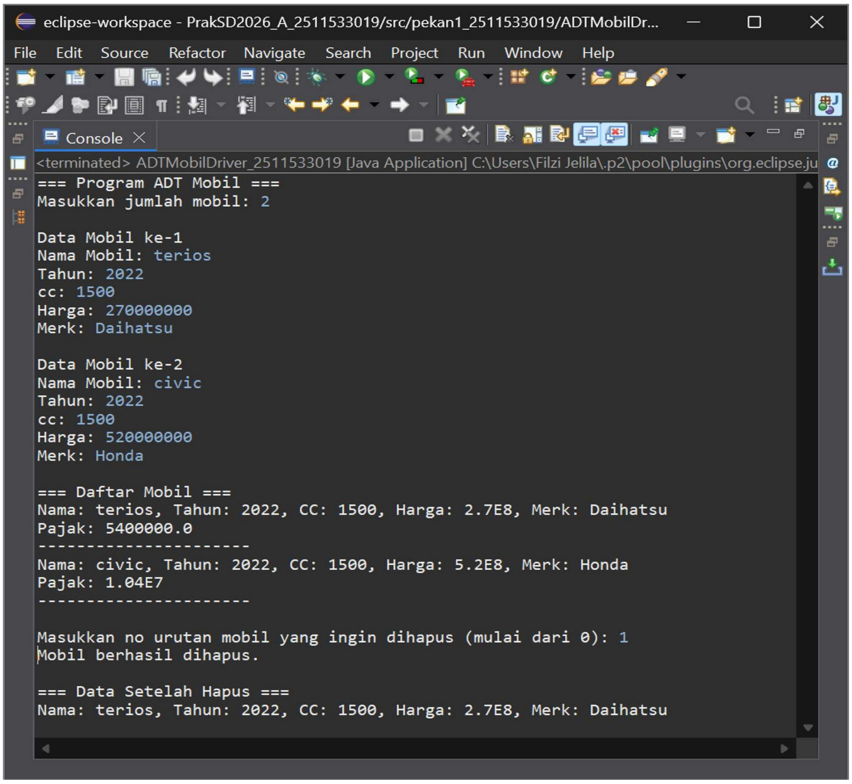
3) Run Program

Setelah dirun program akan menampilkan output seperti berikut.

```
Console
ADTMobilDriver_2511533019 [Java Application] C:\Users\Filz
=== Program ADT Mobil ===
Masukkan jumlah mobil:
```

User akan diminta untuk menginputkan beberapa hal seperti banyak, nama, tahun, cc, merk mobil dan lain sebagainya.

Apabila sudah selesai keseluruhan output akan seperti berikut.



# BAB III
# PENUTUP

## 3.1 Kesimpulan

Pada praktikum kali ini, saya telah mempelajari dan menerapkan konsep Abstract Data Type (ADT) dalam pemrograman Java melalui dua studi kasus, yaitu ADT Jam pada praktikum sebelumnya dan ADT Mobil pada tugas praktikum ini.

Pada praktikum ADT Jam, saya memahami dasar dalam mendefinisikan sebuah objek yang memiliki atribut serta operasi yang berkaitan dengan pengolahan waktu. Selanjutnya, pada praktikum ADT Mobil, pemahaman tersebut dikembangkan dengan studi kasus yang lebih kompleks, yaitu pengelolaan data mobil yang melibatkan penyimpanan banyak objek menggunakan struktur data ArrayList.

Melalui kedua praktikum ini, saya semakin memahami konsep pemrograman berorientasi objek (OOP), seperti pembuatan class, penggunaan konstruktor, serta penerapan enkapsulasi melalui method getter dan setter. Selain itu, saya juga memahami perbedaan peran antara class ADT sebagai blueprint objek dan class driver sebagai pengendali jalannya program.

Penerapan operasi seperti menambah, menampilkan, dan menghapus data pada kedua studi kasus menunjukkan bahwa konsep ADT sangat membantu dalam mengelola data secara terstruktur, modular, dan lebih mudah dikembangkan. Dengan demikian, praktikum ini memberikan pemahaman yang lebih mendalam serta menjadi dasar penting dalam pembuatan program yang lebih kompleks di masa mendatang.

## DAFTAR PUSTAKA

[1] Wahyudi, "*Modul Praktikum Pekan1"* PowerPoint slides, Praktikum Algoritma dan pemrograman, Universitas Andalas, Padang, 2025.

[2] Wahyudi Departemen Informatika Universitas Andalas, "Abstract Data Type (ADT)," Power Point Slides, Mata Kuliah Struktur Data, Padang, 2026.

[3] Wahyudi Departemen Informatika Universitas Andalas, "ArrayList," Power Point Slides, Mata Kuliah Struktur Data, Padang, 2026.

[4] Oracle, "The Java™ Tutorials: Classes and Objects," 2024. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/concepts/. [Accessed: Mar. 31, 2026]

[5] Oracle, "Collections Framework," 2024. [Online]. Available: https://docs.oracle.com/javase/tutorial/collections/. [Accessed: Mar. 31, 2026].